

Digital Nurture - Design Patterns and Principles HandsOn

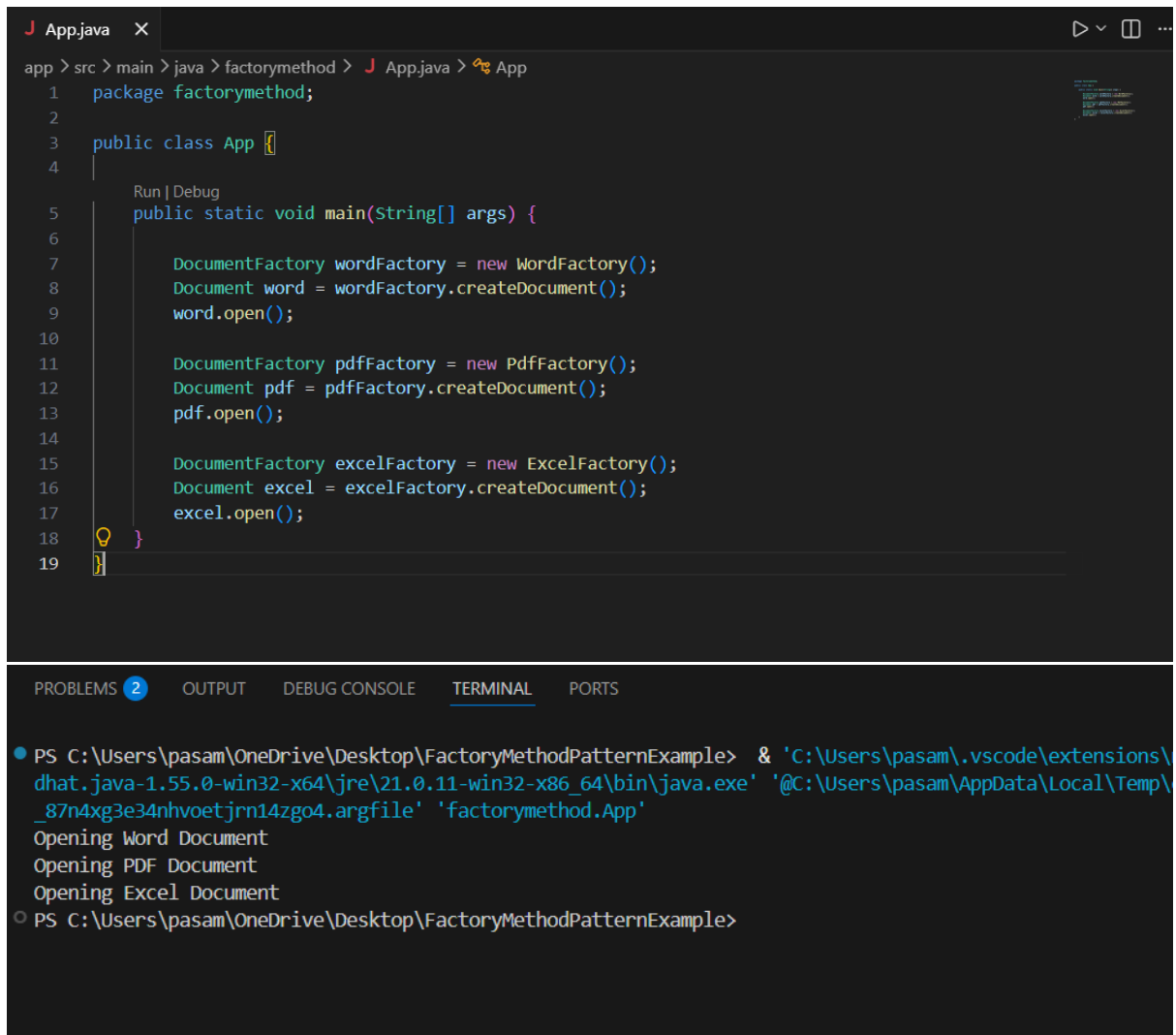
Exercise 1: Implementing the Singleton Pattern

```
App.java X Logger.java Student.java
app > src > main > java > javapractice > App.java > App
1 package javapractice;
2
3 public class App {
4
5     Run | Debug
6     public static void main(String[] args) {
7
8         Logger logger1 = Logger.getInstance();
9         Logger logger2 = Logger.getInstance();
10
11         logger1.log(message: "Application Started");
12         logger2.log(message: "User Logged In");
13
14         System.out.println(logger1 == logger2);
15     }
16 }
```

```
App.java X Logger.java Student.java
app > src > main > java > javapractice > Logger.java > Logger
1 package javapractice;
2
3 public class Logger {
4
5     private static Logger instance = new Logger();
6
7     private Logger() {
8         System.out.println("Logger object created.");
9     }
10
11     public static Logger getInstance() {
12         return instance;
13     }
14
15     public void log(String message) {
16         System.out.println("LOG: " + message);
17     }
18 }
```

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\pasam\Documents\JavaPractice> & 'C:\Users\pasam\.vscode\extensions\redhat.
win32-x86_64\bin\java.exe' '@C:\Users\pasam\AppData\Local\Temp\cp_55hs9szpssthpmqnr5tg
Logger object created.
LOG: Application Started
LOG: User Logged In
true
PS C:\Users\pasam\Documents\JavaPractice>
```

Exercise 2: Implementing the Factory Method Pattern



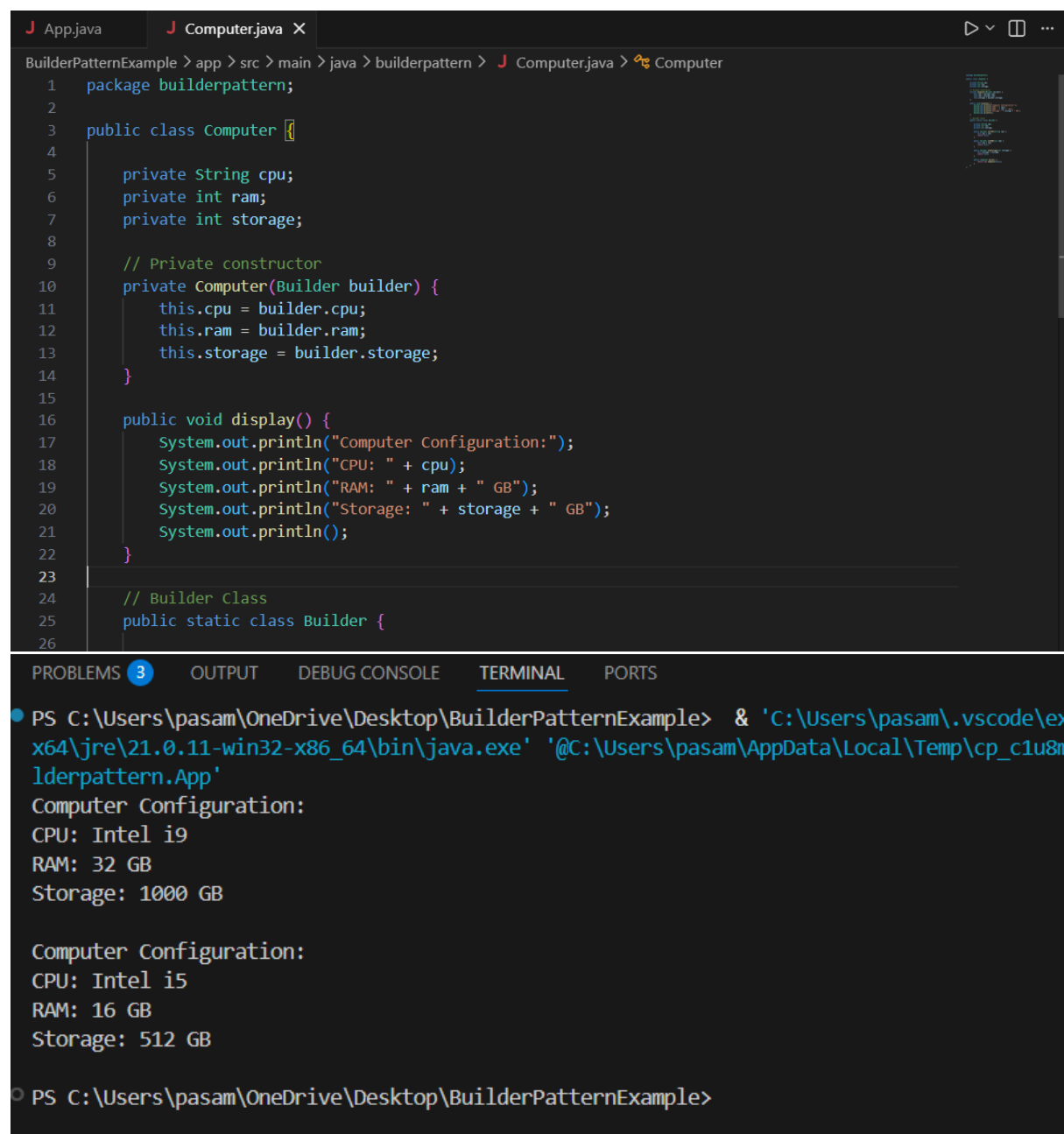
The screenshot shows a VS Code editor window with a file named `App.java` open. The code implements the Factory Method Pattern. It defines a `DocumentFactory` interface and three concrete implementations: `WordFactory`, `PdfFactory`, and `ExcelFactory`. The `App` class has a `main` method that creates instances of these factories and uses them to create and open documents.

```
1 package factorymethod;
2
3 public class App {
4
5     public static void main(String[] args) {
6
7         DocumentFactory wordFactory = new WordFactory();
8         Document word = wordFactory.createDocument();
9         word.open();
10
11         DocumentFactory pdfFactory = new PdfFactory();
12         Document pdf = pdfFactory.createDocument();
13         pdf.open();
14
15         DocumentFactory excelFactory = new ExcelFactory();
16         Document excel = excelFactory.createDocument();
17         excel.open();
18     }
19 }
```

The terminal output shows the execution of the program, which prints the following messages:

```
PS C:\Users\pasam\OneDrive\Desktop\FactoryMethodPatternExample> & 'C:\Users\pasam\.vscode\extensions\dhat.java-1.55.0-win32-x64\jre\21.0.11-win32-x86_64\bin\java.exe' '@C:\Users\pasam\AppData\Local\Temp\_87n4xg3e34nhvoetjrn14zgo4.argfile' 'factorymethod.App'
Opening Word Document
Opening PDF Document
Opening Excel Document
PS C:\Users\pasam\OneDrive\Desktop\FactoryMethodPatternExample>
```

Exercise 3: Implementing the Builder Pattern



```
BuilderPatternExample > app > src > main > java > builderpattern > Computer.java > Computer

1 package builderpattern;
2
3 public class Computer {
4
5     private String cpu;
6     private int ram;
7     private int storage;
8
9     // Private constructor
10    private Computer(Builder builder) {
11        this.cpu = builder.cpu;
12        this.ram = builder.ram;
13        this.storage = builder.storage;
14    }
15
16    public void display() {
17        System.out.println("Computer Configuration:");
18        System.out.println("CPU: " + cpu);
19        System.out.println("RAM: " + ram + " GB");
20        System.out.println("Storage: " + storage + " GB");
21        System.out.println();
22    }
23
24    // Builder Class
25    public static class Builder {
26
```

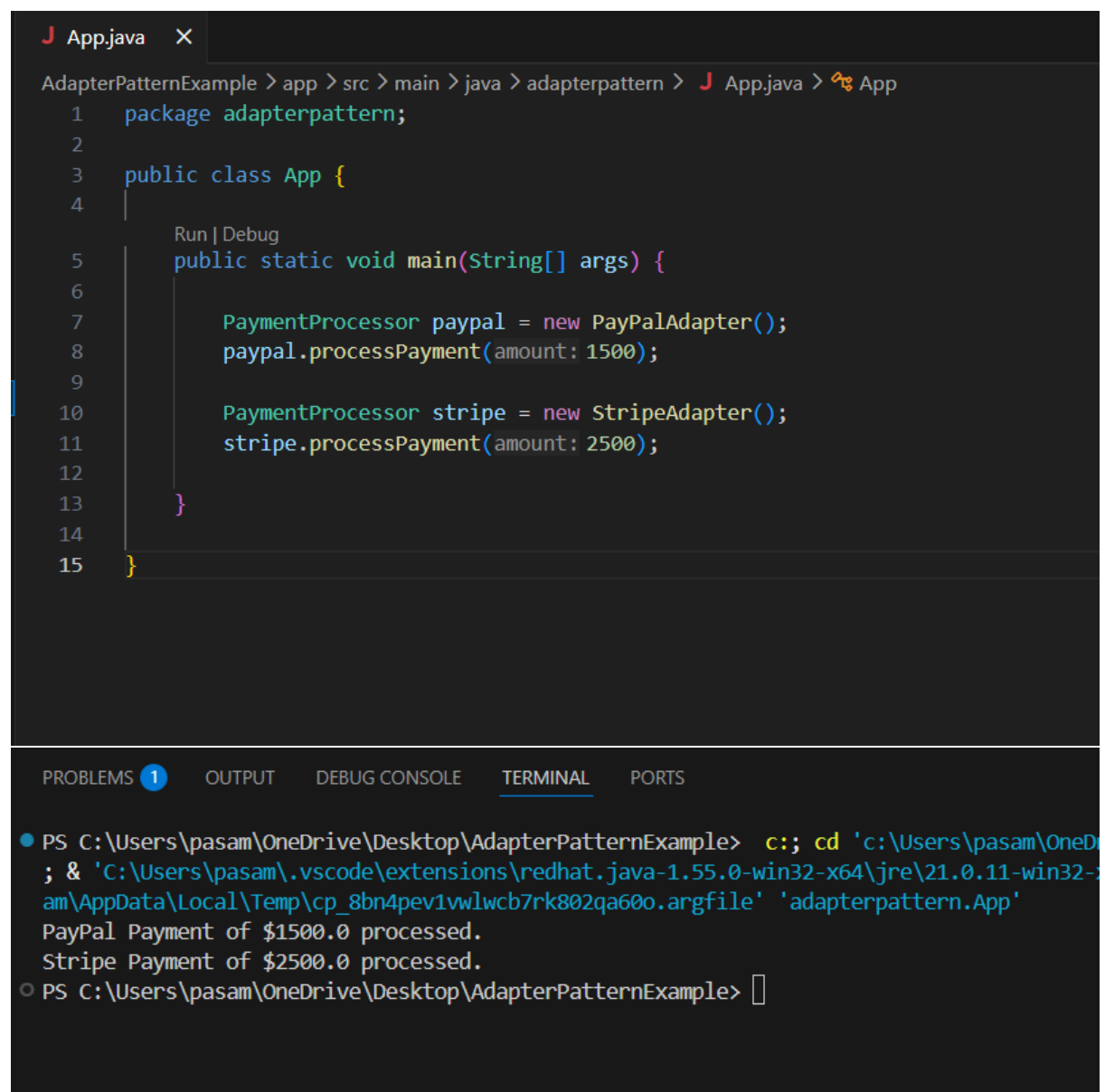
PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\pasam\OneDrive\Desktop\BuilderPatternExample> & 'C:\Users\pasam\.vscode\extensions\ms-java-jre\21.0.11-win32-x86_64\bin\java.exe' '@C:\Users\pasam\AppData\Local\Temp\cp_c1u8n\builderpattern.App'
Computer Configuration:
CPU: Intel i9
RAM: 32 GB
Storage: 1000 GB

Computer Configuration:
CPU: Intel i5
RAM: 16 GB
Storage: 512 GB

PS C:\Users\pasam\OneDrive\Desktop\BuilderPatternExample>
```

Exercise 4: Implementing the Adapter Pattern



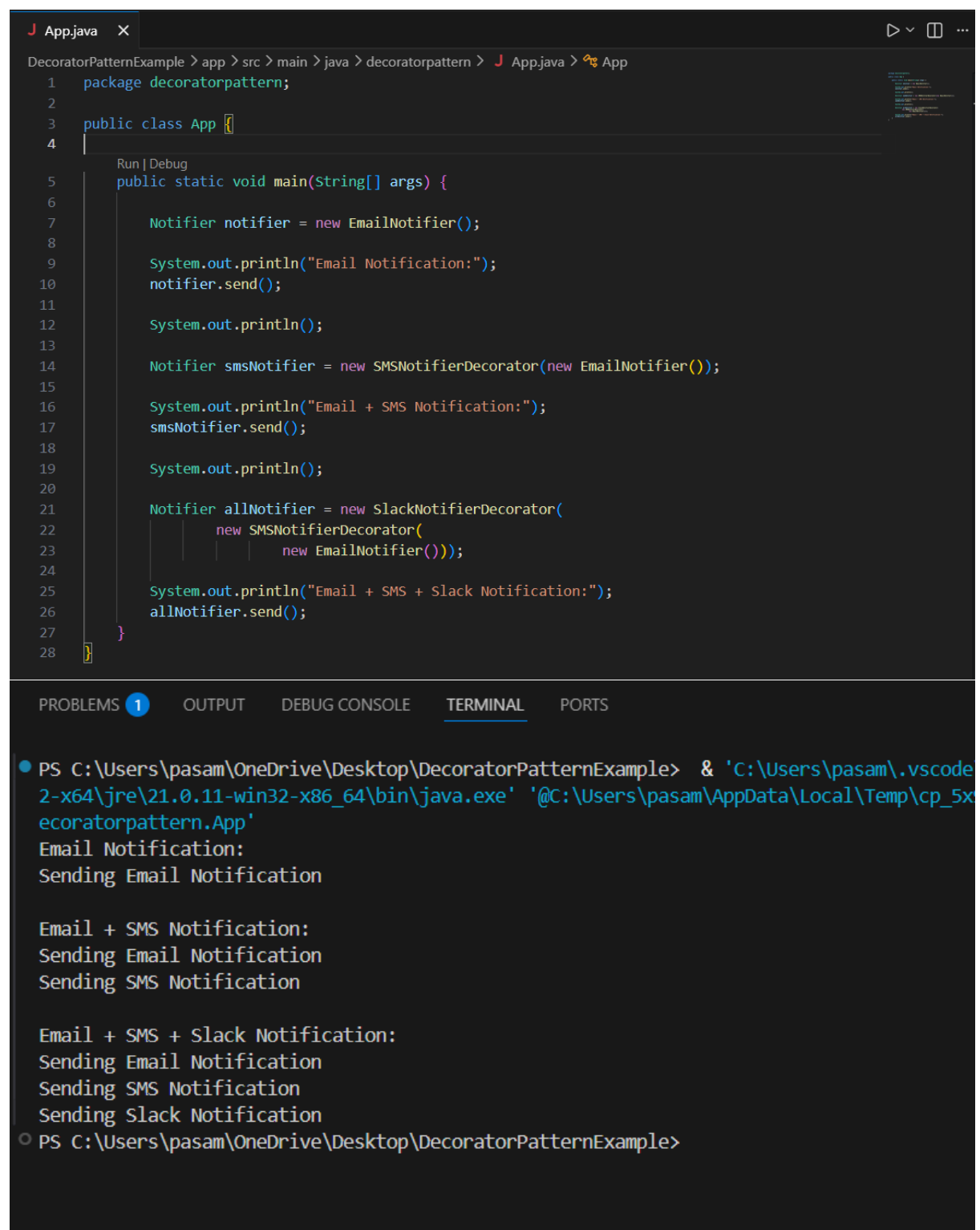
The screenshot shows a VS Code editor window with a file named `App.java` open. The file is located at `AdapterPatternExample > app > src > main > java > adapterpattern`. The code defines a `public class App` with a `main` method. Inside `main`, two `PaymentProcessor` objects are created: `paypal` (using `PayPalAdapter`) and `stripe` (using `StripeAdapter`). Both are then used to process payments of \$1500.0 and \$2500.0 respectively.

```
1 package adapterpattern;
2
3 public class App {
4     Run | Debug
5     public static void main(String[] args) {
6
7         PaymentProcessor paypal = new PayPalAdapter();
8         paypal.processPayment(amount: 1500);
9
10        PaymentProcessor stripe = new StripeAdapter();
11        stripe.processPayment(amount: 2500);
12    }
13 }
14
15 }
```

The terminal output shows the command executed to run the application and the resulting output:

```
PS C:\Users\pasam\OneDrive\Desktop\AdapterPatternExample> c::; cd 'c:\Users\pasam\OneDrive\Desktop\AdapterPatternExample'; & 'C:\Users\pasam\.vscode\extensions\redhat.java-1.55.0-win32-x64\jre\21.0.11-win32-x64\bin\java.exe' -cp 'C:\Users\pasam\AppData\Local\Temp\cp_8bn4pev1vwlwcb7rk802qa60o.argfile' 'adapterpattern.App'
PayPal Payment of $1500.0 processed.
Stripe Payment of $2500.0 processed.
PS C:\Users\pasam\OneDrive\Desktop\AdapterPatternExample>
```

Exercise 5: Implementing the Decorator Pattern



```
App.java X
DecoratorPatternExample > app > src > main > java > decoratorpattern > J App.java > App

1 package decoratorpattern;
2
3 public class App {
4
5     public static void main(String[] args) {
6
7         Notifier notifier = new EmailNotifier();
8
9         System.out.println("Email Notification:");
10        notifier.send();
11
12        System.out.println();
13
14        Notifier smsNotifier = new SMSNotifierDecorator(new EmailNotifier());
15
16        System.out.println("Email + SMS Notification:");
17        smsNotifier.send();
18
19        System.out.println();
20
21        Notifier allNotifier = new SlackNotifierDecorator(
22            new SMSNotifierDecorator(
23                new EmailNotifier()));
24
25        System.out.println("Email + SMS + Slack Notification:");
26        allNotifier.send();
27    }
28 }
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS C:\Users\pasam\OneDrive\Desktop\DecoratorPatternExample> & 'C:\Users\pasam\.vscode\2-x64\jre\21.0.11-win32-x86_64\bin\java.exe' '@C:\Users\pasam\AppData\Local\Temp\cp_5x\decoratorpattern.App'

Email Notification:
Sending Email Notification

Email + SMS Notification:
Sending Email Notification
Sending SMS Notification

Email + SMS + Slack Notification:
Sending Email Notification
Sending SMS Notification
Sending Slack Notification

- PS C:\Users\pasam\OneDrive\Desktop\DecoratorPatternExample>